

Major Project — Custom Vulnerable Web Lab & Automated Exploit Testing Suite

Student Name: Ankan Sarkar

Role: Ethical Hacking & Cybersecurity Intern

Organization: VaultofCodes

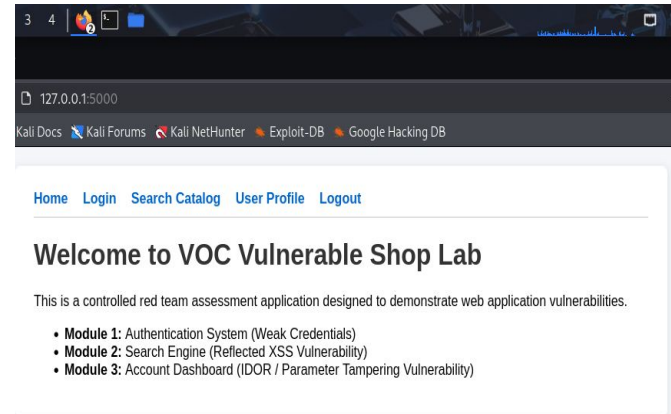
Selected Option: Option 11 — Custom Vulnerable Lab (Advanced Option)

Problem Statement & Project Goals:

- **The Problem:** Insecure coding practices lead to persistent web application vulnerabilities such as broken access control, XSS, and weak authentication in production systems.
- **Project Objective:** Build a complete, functional lab environment (`app.py`) featuring multiple controlled vulnerabilities alongside an automated exploit test suite (`exploit_test.py`).
- **Value Delivered:** Demonstrates end-to-end vulnerability execution, payload verification, and defensive remediation strategies.

Lab Architecture & Vulnerability Modules:

- **Module 1 — Weak Authentication:** Uses predictable default credentials (`admin` / `admin123`) without rate limiting.
- **Module 2 — Reflected XSS (`/search?q=`):** Echoes dynamic URL parameters directly into rendered HTML without output encoding.
- **Module 3 — IDOR / Parameter Tampering (`/profile?user_id=`):** Evaluates account access based on client-controlled URL parameters rather than server-side session tokens.



Exploit Demonstration & Key Findings:

- **Automated Proof-of-Concept:** Executed `exploit_test.py` against active target server <http://127.0.0.1:5000> .
- **Reflected XSS Verified:** Payload `<script>alert(...)` rendered in DOM.
- **Privilege Escalation Verified:** Tampered `?user_id=3` to `?user_id=1` to extract private Administrator API key (`SECRET_API_KEY_ADMIN_9901`).
- **Exploit Result:** 100% automated verification across all three attack vectors.

```
root@kali: /home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
File Actions Edit View Help

root@kali:~/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
└─ python3 app.py
[+] Starting Vulnerable Web Application on http://127.0.0.1:5000
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
[+] Starting Vulnerable Web Application on http://127.0.0.1:5000
* Debugger is active!
* Debugger PIN: 684-397-809
127.0.0.1 - - [10/Aug/2026 19:29:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET /search?q=<script>alert('XSS_VULNERABILITY_CONFIRMED')</script> HTTP/1.1" 200 -
127.0.0.1 - - [10/Aug/2026 19:29:47] "GET /profile?user_id=1 HTTP/1.1" 200 -
```

```
root@kali: /home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
File Actions Edit View Help

root@kali:~/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
└─ python3 exploit_test.py

VOC Vulnerable Lab - Automated Exploit Suite

[+] Testing Module 1: Weak Authentication Exploitation
[SUCCESS] Compromised Credentials: admin:password

[+] Testing Module 2: Reflected Cross-Site Scripting (XSS)
[SUCCESS] Reflected XSS Payload rendered unsanitized in DOM!
Target URL: http://127.0.0.1:5000/search?q=<script>alert('XSS_VULNERABILITY_CONFIRMED')</script>

[+] Testing Module 3: Insecure Direct Object Reference (IDOR) / Parameter Tampering
[SUCCESS] IDOR Privilege Escalation Confirmed: Extracted Admin Data:
- Role: Administrator
- Extracted API Key: SECRET_API_KEY_ADMIN_9901
- Extracted Compensation Data: $150,000

ASSESSMENT SUMMARY RESULT
Weak Authentication Vulnerability: VULNERABLE
Reflected XSS Vulnerability: VULNERABLE
IDOR / Parameter Tampering: VULNERABLE

root@kali:~/home/kali/Desktop/VaultofCodes/Training & Internship/Assignments/Final Assignment/MajorProject
```

Remediation & Secure Coding Recommendations:

- **Enforce Context-Aware Output Encoding:** Apply `htmlspecialchars()` or jinja auto-escaping to eliminate Reflected XSS.
- **Server-Side Session Validation:** Verify user permissions against secure server sessions (`$_SESSION['user_id']`) to block IDOR attacks.
- **Credential Hardening:** Implement multi-factor authentication (MFA) and account lockouts after failed login attempts.